

Spend Smart: An AI-Powered Personal Finance Management Platform for Budgeting, Analytics, and Automated Financial Assistance

Aniket Nimiya

Dept. CSE(AI&ML), Krishna Institute of Engineering & Technology (KIET), Ghaziabad, Delhi-NCR, Uttar Pradesh, India,
aniket.2226cseml1022@kiet.edu

Aryan

Dept. CSE(AI&ML), Krishna Institute of Engineering & Technology (KIET), Ghaziabad, Delhi-NCR, Uttar Pradesh, India,
aryan.2226cseml1029@kiet.edu

Aman Chauhan

Dept. CSE(AI&ML), Krishna Institute of Engineering & Technology (KIET), Ghaziabad, Delhi-NCR, Uttar Pradesh, India,
aman.2226cseml1120@kiet.edu

Dipendra Yadav

Dept. CSE(AI&ML), Krishna Institute of Engineering & Technology (KIET), Ghaziabad, Delhi-NCR, Uttar Pradesh, India,
dipendra.2226cseml11@kiet.edu

Under the guidance of Ms. Surbhi Verma

Assistant Professor, Dept. CSE(AI&ML), Krishna Institute of Engineering & Technology (KIET), Ghaziabad, Delhi-NCR, Uttar Pradesh, India,
surbhi.verma@kiet.edu

Abstract—The new generation of personal finance management apps should go beyond manual accounting and become more sophisticated in terms of providing intelligent assistance, automatization, and visibility regarding financial matters. In this paper, I propose Spend Smart an all-in-one full-stack web app that allows individual users to record their income and expenses, categorize the transactional data, establish monthly budgets, save money, and get insightful conclusions about their own financial behavior based on interactive dashboard. The proposed technology includes Next.js and React frontend, secure validation of financial data with React Hook Form and Zod, Prisma-based communication with database, as well as Supabase for storing files in the cloud. The protection of users' financial accounts and credentials can be achieved with Clerk authentication and Arcjet based abuse mitigation solutions. For better decision support, the platform also utilizes the power of artificial intelligence through Google services to analyze the spending habits of each individual user and give advice accordingly. The recharts tool will help visualize financial statistics interactively, and Inngest will manage background tasks such as monthly analyses and reports. Resend and React Email will allow scheduling reminders about financial issues such as budgeting and monthly summaries. In conclusion, the paper views the proposed Spend Smart application as an actual embodiment of an innovative idea of a digital personal finance assistant which can be obtained by integrating a variety of advanced web technologies into the platform of interest.

Keywords— Personal finance, financial analytics, web application, artificial intelligence, budgeting, automation, expense tracking

I. INTRODUCTION

Personal finance management tools based on digital technologies have progressed from applications primarily devoted to the recording of transactions toward more complex software that allows for budgeting, forecasting, analysis of behavior patterns, and proactive advice. On the other hand, recent studies in the

field of AI-based personal finance trackers demonstrate the growing popularity of applications capable not only of storing information about financial transactions but also analyzing behavioral patterns and making spending decisions for the user. Conventional expense trackers usually involve multiple repetitions of inputting the data and giving feedback after the transactions took place. Such a mechanism is of little help for people whose needs include real time access to information about their financial state, control over budgeting, and habit modification. According to research materials on financial dashboards, personal finance systems should be designed with visualization and contextual reporting in mind. Furthermore, ease of use of the interface is an important requirement to ensure maximum user satisfaction. For this purpose, Spend Smart is intended as a solution to all the above-mentioned problems. Unlike traditional financial managers, it incorporates financial tracking, data verification, automation, security, visualization, and AI-based advice into a single web application. The purpose of the system is to allow users to comprehend both what money was spent on and the meaning of their financial pattern and how to change it for the better.

II. RELATED WORK AND MOTIVATION

Published and publicly available works related to personal finance systems consistently point out three key issues: fragmentation of accounting, lack of user engagement, and insufficiency of data analysis. A recently published study on the simplification of the process of personal finance management emphasizes the fact that many users still rely on a non-systemic approach and manual entry. As a result, insight creation becomes difficult. At the same time, descriptions of new AI-powered finance tracking software

note that traditional apps are rarely able to provide any predictive or advisory features beyond categorization and data summarization. Research on finance dashboards shows that users can understand their financial data effectively when information is presented by using trends, comparisons, and key highlights instead of just listing down transactions. While designing the dashboard for our project Spend Smart, our main focus on maintaining a clear information hierarchy, interactive elements, and concise data representation to improve usability. Studies on expense tracking applications also indicate that users prefer fast data entry, proper categorization, and timely feedback rather than having too many complex features. Our system was designed with two main goals. First, to provide a single platform where users can manage all their financial activities efficiently. Second, to go beyond basic record keeping by providing AI-based insights, automated monthly reports, and visual representations of financial data.

Spend Smart is not just an ordinary expense tracker but designed as a smart financial assistant that helps users to understand and improve their financial habits.

III. SYSTEM OBJECTIVES

Five main goals guide the design of Spend Smart:

- To make it easy to record income and expenses by using multiple account category and reducing manual efforts.
- To help with monthly budgeting, keeping track of savings, and managing transactions made by the user.
- To use AI integration helping user to get insights of their spendings.
- To automate tasks that happen over and over again in finance, like monthly summaries, alerts, and analytical jobs that happen on a regular basis through recurring feature.
- To maintain user trust through secure authentication, controlled file storage, and protection against suspicious requests.

These objectives reflect both the expectations found in current personal finance research and the functional sections commonly used in technical system papers. Together, they define *Spend Smart* as a user-centered, analytics-oriented web platform rather than a simple data-entry utility.

IV. PROPOSED SYSTEM ARCHITECTURE

Spend Smart is a full-stack system architecture. Its frontend, made in and React, includes all the elements needed for a user to perform daily actions: log transactions, create budgets, navigate the dashboard, access reporting tools, upload assets, and others. In order to provide users with a ease experience, Spend Smart uses shaden, elements to design frontend modues. Meanwhile, React Hook Form and Zod check for the data validation and processing the data provided by users.

In the data section, queries and schema updates become much easier through Prisma. In addition, Supabase allows managing files and documents uploaded by the users independently from transaction data, thus keeping them both organized and backed by the cloud.

As already mentioned, security plays a key role when handling any

kind of financial data. Therefore, in addition to implementing encryption, the proposed architecture uses Clerk, a platform authorization. Another essential part of any system that handles sensitive data is abuse detection that is implemented via Arcjet.

The difference between Spend Smart and ordinary financial recorders is its intelligent component. Using Google-based AI services, Spend Smart analyzes the spending behavior of users and provides them with individualized advice. Moreover, the platform performs monthly analysis and generates financial reports using Inngest, a system responsible for scheduling background jobs. Recharts is another component that helps visualize spending in various categories and compare it with user's monthly income and budget. Finally, Spend Smart uses Resend and React Email to perform automated monthly reporting, including emails with budget updates and notifications.

V. METHODOLOGY AND DESIGN APPROACH

The development strategy of Spend Smart follows a feature-driven approach based on user needs and requirements. The process started by identifying common tasks performed in financial applications, such as recording income and expenses, categorizing transactions, monitoring budgets, analyzing trends, and generating financial insights. These features were selected because they directly improve the usability and effectiveness of latest finance systems. While designing the frontend of our project Spend Smart, our main focus was on to create a simple, responsive, and user-friendly interface that allows users to interact with the application easily. Since financial applications depend heavily on accurate user input, proper data validation is essential. For this purpose, React Hook Form and Zod are used to validate inputs, reduce errors, and ensure data consistency.

For analysis, user transactions are organized into categories and evaluated based on different factors such as time period and budget limits. The processed data is then displayed through interactive dashboards along with AI-based insights, helping users better understand their financial activities.

This methodology is consistent with recent trends in AI-based finance tracking where similar approaches are employed to boost awareness of users and make personalized finance management more effective.

VI. SYSTEM ARCHITECTURE

A. Architectural Overview

Architectural Overview Spend Smart uses a layered or full-stack architecture, consisting of four main layers: Presentation Layer deals with the rendering and interaction part of the system Application Logic Layer contains routes, handling requests and responding, external services Data Layer – stores data used by the application Automation Layer executes background tasks The layered architecture helps developers work independently on each layer and scale them accordingly.

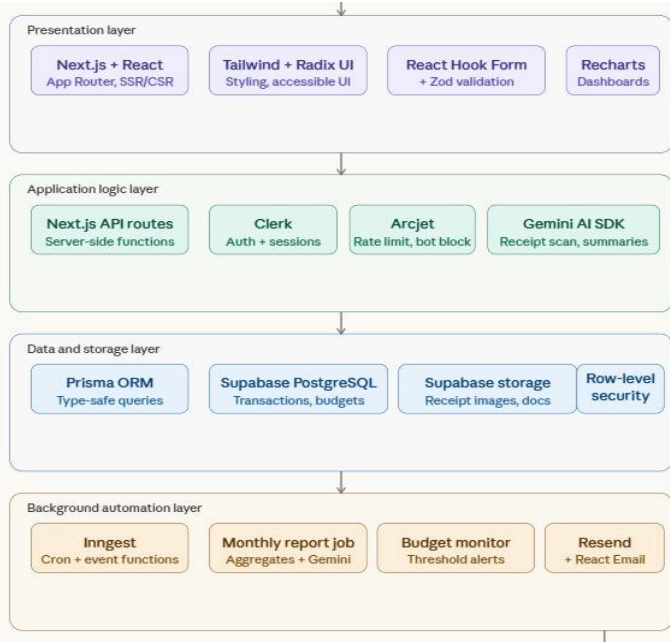
Presentation Layer Built entirely with React & Next.js, with Next.js App Router providing file-based routing and Server Components that render on the server for increased performance and Client

Components. Tailwind CSS implements a utility-first approach for styles, and Radix UI offers accessible components for complicated UI elements like dialogs, dropdown menus and popovers.

Application Logic layer routes containing API handlers are written with Next.js server-side methods. They interact with the database using Prisma ORM and request to external services including Clerk, Arcjet and Gemini AI API (for receipt scanning). All incoming data validation is done by zod before storing to the database, to guarantee data integrity. Data and Storage Layer Uses a PostgreSQL database managed with Supabase and Prisma, to perform queries.

Images and receipts are stored in buckets in Supabase Cloud Storage, whereas financial data (transactions, budgets and user information) is stored in the database.

Background Automation Layer Runs event-driven and scheduled functions using Inngest. Tasks include generating financial reports on a monthly basis, analysing transactions regularly, sending budget consumption alerts and sending emails.



B. Data Flow Diagram

The following describes the primary data flow within the application:

- 1) User authenticates through Clerk to log into the system and land on the dashboard.
- 2) The dashboard retrieves transaction summary information, budget usage and chart data from the database, using Prisma.
- 3) Once a user submits a receipt, the receipt image is saved to Supabase Storage bucket, after which the URL of the receipt image is sent to the Gemini AI API for analysis.
- 4) The gemini API analyses the merchant name, date of transaction, individual line items and overall transaction amount, sending back a JSON object that populates the new transaction form fields.
- 5) After confirmation by the user, transaction details are validated by Zod and persisted to the database using Prisma.
- 6) Every month-end, an event-driven function executed by Inngest queries the database, generates a summary of finances, sends the

results to Gemini API for natural language generation and sends the email using Resend and a React Email template.

C. Security Architecture

Security Architecture Security features exist in multiple layers of the application. Clerk provides the authentication layer with OAuth and email login, as well as session management. An additional middleware provided by Arcjet implements rate limiting and bot detection capabilities, blocking suspicious request patterns before reaching the Application Logic layer. All incoming data is validated using Zod schemas to reject malformed data. Supabase Row Level Security policies ensure that user access is restricted to own financial data only.

VII. TECHNOLOGY STACK

A. Frontend Technologies

Next.js and React make up the core technologies of our front-end stack. We chose Next.js due to its ability to provide hybrid rendering – SSR (server-side rendering) for SEO purposes and for faster page load at the very beginning, as well as client-side rendering that is essential for creating responsive interactive components. React’s components used to create and reuse modular components formed financial dashboards. All styling is performed using Tailwind CSS across the application. CSS files, shaden components providing fast development of user interfaces while maintaining consistency in our design. Radix UI provides primitive components with accessibility and no styles at all including such components as dropdowns, popups, tooltips, and switches, styled by Tailwind CSS. Lucide is an iconography system that provides all sorts of clean and consistent icons for Spend Smart. We use them throughout the application for navigation, financial forms, and dashboard widgets. Recharts is the graph library that is responsible for creating all visualizations of transaction made in Spend Smart made by the user over the month using bar and pie charts.

B. Form Handling and Validation

React Hook Form handles all form state management inside our application. It includes financial transaction entry forms, budget setup forms, financial accounts setup, and receipt uploading forms. Uncontrolled component approach of React Hook Form helps us handle large multi-field forms in a performant way – minimizing unnecessary re-renders of these components. Zod is our validation library responsible for ensuring that financial data entered into forms is consistent and semantically correct. Each transaction, budget, and user settings are validated using Zod prior to executing any database operation.

C. Backend and Data Management

Prisma ORM acts as a data access layer, giving us safe access to underlying database managed by Supabase (PostgreSQL). Prisma schema is responsible for defining all the entities such as User, Account, Transaction, Budget, and Recurring Schedule, as well as setting up relationships between them. Our database migrations are managed by Prisma’s migration engine, allowing for easy management of our database evolution. Supabase not only provides our database but also cloud object storage. Images received from

user receipts are stored using Supabase’s storage buckets with the help of signed URLs. Supabase’s ability to combine both storage and databases in one single platform helps us streamline our infrastructure management while keeping its reliability high.

D. Authentication and Security

Clerk provides all the functionality required for user authentication in Spend Smart. It includes user sign-up, sign-in, OAuth providers, multi-factor authentication, session management, and other services related to users’ safety online. Customizable React components of Clerk provide us with ready-to-use authentication pages that can easily be customized in case of our needs. Arcjet middleware for Next.js allows us to protect our application from any bot traffic, API misuse, and brute-force attacks. Rate limiting rules for different parts of our backend logic are set individually for each specific route. We set stricter limits to receipt uploading and financial reports generating routes.

E. Artificial Intelligence Integration

Artificial Intelligence Integration Spend Smart makes use of Gemini generative artificial intelligence provided by Google as a part of its Generative AI SDK. There are two major applications of Google Gemini in Spend Smart: Receipt Scanning and Data Extraction: Once the user uploads an image of his or her receipt, the image gets encoded into base64-encoded payload and sent to gemini-1.5-pro model along with structured prompt asking to extract the merchant’s name, transaction date, line items, total taxes, and total amount of money spent. As a result, we receive a JSON object from Gemini containing extracted information, which is parsed and used to auto-fill the transaction entry form. It saves time of manually entering this information in Spend Smart. Monthly Financial Summaries: Once per month, on the first day of it, an Inngest-triggered background function extracts financial information, categorizes transactions and budget consumption, and formats them into structured prompt. This information gets submitted to Gemini which generates a helps us handle large multi-field forms in a performant way – minimizing unnecessary re-renders of these components. Zod is our validation library responsible for ensuring that financial data entered into forms is consistent and semantically correct. Each transaction, budget, and user settings are validated using Zod prior to executing any database operation.

F. Backend and Data Management

Prisma ORM acts as a data access layer, giving us safe access to underlying database managed by Supabase (PostgreSQL). Prisma schema is responsible for defining all the entities such as User, Account, Transaction, Budget, and Recurring Schedule, as well as setting up relationships between them. Our database migrations are managed by Prisma’s migration engine, allowing for easy management of our database evolution. Supabase not only provides our database but also cloud object storage. Images received from user receipts are stored using Supabase’s storage buckets with the help of signed URLs. Supabase’s ability to combine both storage and databases in one single platform helps us streamline our infrastructure management while keeping its reliability high.

G. Authentication and Security

Clerk provides all the functionality required for user authentication in Spend Smart. It includes user sign-up, sign-in, OAuth providers, multi-factor authentication, session management, and other services related to users’ safety online. Customizable React components of Clerk provide us with ready-to-use authentication pages that can easily be customized in case of our needs. Arcjet middleware for Next.js allows us to protect our application from any bot traffic, API misuse, and brute-force attacks. Rate limiting rules for different parts of our backend logic are set individually for each specific route. We set stricter limits to receipt uploading and financial reports generating routes.

H. Artificial Intelligence Integration

Artificial Intelligence Integration Spend Smart makes use of Gemini generative artificial intelligence provided by Google as a part of its Generative AI SDK. There are two major applications of Google Gemini in Spend Smart: Receipt Scanning and Data Extraction: Once the user uploads an image of his or her receipt, the image gets encoded into base64-encoded payload and sent to gemini-1.5-pro model along with structured prompt asking to extract the merchant’s name, transaction date, line items, total taxes, and total amount of money spent. As a result, we receive a JSON object from Gemini containing extracted information, which is parsed and used to auto-fill the transaction entry form. It saves time of manually entering this information in Spend Smart. Monthly Financial Summaries: Once per month, on the first day of it, an Inngest-triggered background function extracts financial information, categorizes transactions and budget consumption, and formats them into structured prompt. This information gets submitted to Gemini which generates a personalized financial summary narrative that contains user spending analysis, areas that require more attention, comparison with previous months’ data, and recommendations. It gets sent to users via monthly financial summary email report.

I. Background Automation

Inngest is used to run all automated background tasks in Spend Smart. Inngest functions are event-driven serverless tasks that operate outside our request-response cycle to avoid blocking the user interface. The most important Inngest functions include the following: a monthly cron function that triggers on the first day of each month, aggregates financial data, submits it to Gemini to create a summary, and sends a monthly email report through Resend. There is also a recurring transactions processor that generates recurring transaction records once a month based on subscriptions and bills set up by the user. Additionally, budget threshold monitors send budget alert emails if spending in specific categories exceeds the budget limit set by the user. Inngest provides execution logs for all background tasks, enabling us to track their performance in real time.

J. Email Communication

Resend is the email delivery service that sends all the transactional and report emails from Spend Smart. It offers reliable delivery along with email analytics for tracking delivery status and opening rates. React Email is a library that gives us tools for creating email templates. It allows us to design email templates like building React

components. These templates include conditional statements, component hierarchy, and options for customizing design and content based on properties. All email templates in Spend Smart are created with React Email and rendered on the server before being sent out.

VIII. KEY FEATURES AND WORKFLOWS

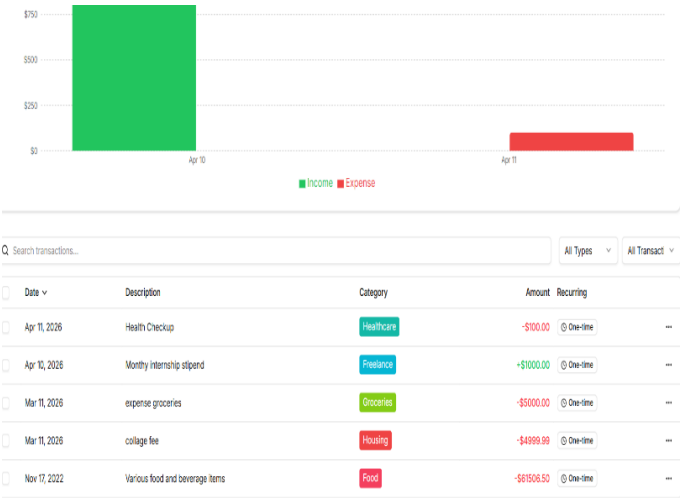
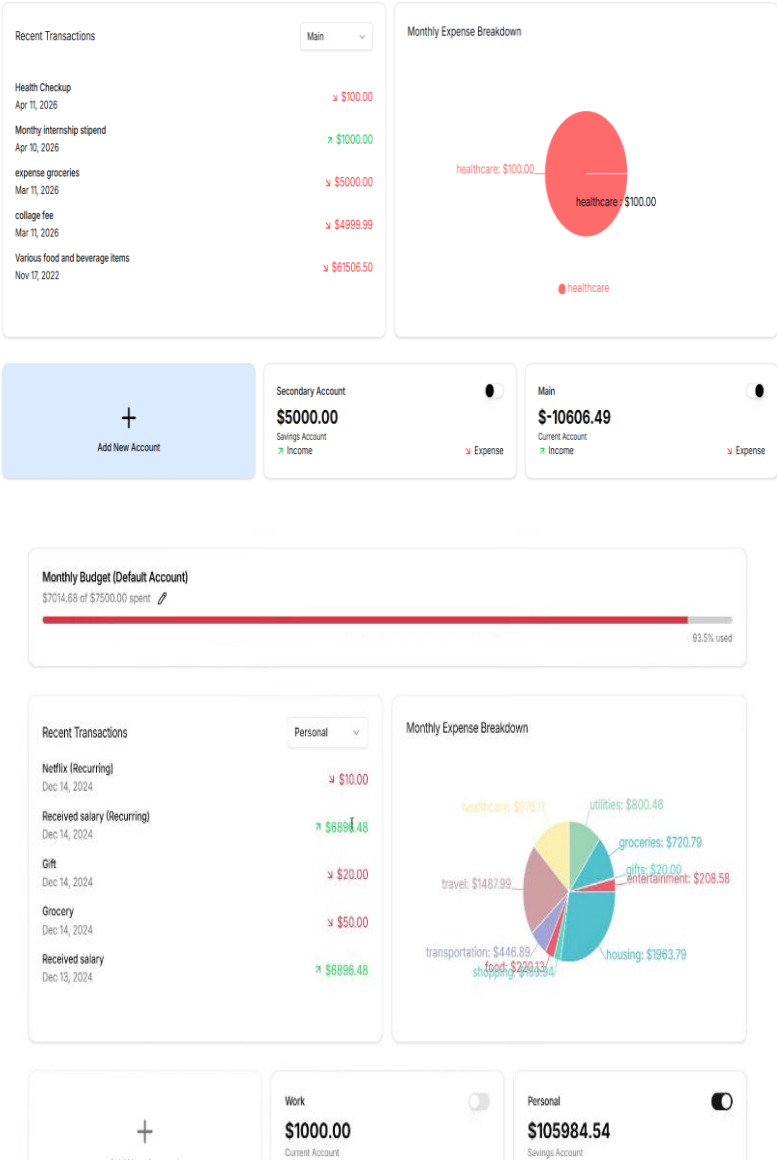
A. Interactive Financial Dashboard

The main component of our application is its dashboard. As soon as the user logs in, they gain access to the real-time view of their financial standing, including their total income, total expenses, net savings, and budget consumption. Recharts creates four key charts on the dashboard

- A bar chart illustrating total income vs total expenses by month,
- Pie chart depicting total spending by category
- Line graph depicting balance in savings, and
- Progress bars indicating present budget consumption against set limits.

All chart data comes from the server using server components provided by Next.js, meaning that the dashboard loads with prerendered charts and not through client-side fetch waterfalls.

Responsive charts and interactive tooltips guarantee proper functionality across both desktop and mobile interfaces.



B. Transaction Management

Users may create transactions either manually via a transaction form that prompts for merchant name, transaction amount, category, date, notes, and account.

For the creation of transactions in a performant manner, the form uses React Hook Form, while Zod takes care of input validation. Alternatively, users may upload an image of a receipt and the Gemini workflow kicks off.

Extracted transaction details populate the form automatically, allowing the user to review the transaction, make any necessary adjustments and confirm it.

Compared to traditional budgeting applications, the process of adding an entry becomes significantly simpler due to artificial intelligence assistance.

C. Budget Management

Users may define monthly budgets for arbitrary numbers of expense categories, e.g., food, transport, entertainment, and utilities.

The system monitors the amount spent on categories and reports it to the user in real time by means of progress bars in the budget section on the dashboard.

Once the amount spent crosses a configured threshold (say, 80).

D. Recurring Transactions

Spend Smart provides users the ability to configure recurring payments, subscriptions, and income such as salary. Each transaction may be specified with an amount, frequency , category, and a start date. After that, the function runs Inngest scheduled task that executes automatically at the configured interval to create a transaction.

As a result, there will never be a missed bill or subscription because they will always appear as transactions in the financial record of the user.

E. AI Monthly Financial Reports

At the end of every calendar month, the system automatically creates an AI-based report for every single user. The Inngest monthly cron job collects information about transactions during the month and performs several operations, i.e., computes category amounts, finds the percentage of the saved money (savings rate), and compares results with those from the previous month. Structured data is sent to Gemini along with the prompt to generate a comprehensive human-readable report. In order to do this, Gemini uses the provided data and generates a report which gets added to a React Email template together with metrics and graphs. Finally, the email goes out via Resend service.

IX. EVALUATION AND DISCUSSION

A. Performance

Next.js server components and route-level caching ensure that the dashboard and transaction pages load with minimal client-side rendering overhead. Prisma’s query optimizer generates efficient SQL, and database indexes on commonly queried fields such as user ID, transaction date, and category reduce query latency. The use of Inngest for asynchronous processing ensures that AI-intensive operations—such as receipt scanning and report generation—do not introduce latency into the synchronous user experience.

B. Security Assessment

The multi layered security model of Spend Smart effectively deals with the common web application vulnerabilities. used Clerk’s JWT-based for authentication enable the login through google email id prevents unauthorized access through invalid credentials. created the middlewares which restrict the unauthorized access to the protected route. Arcjet middleware blocks automated bot traffic and filter the traffic visiting website enforces per-user rate limits, mitigating denial of service attack. Zod’s strict schema validation at all API boundaries prevents injection-style attacks through malformed financial inputs. Supabase row-level security ensures strict data isolation between users.

C. Usability and User Experience

The application is designed keeping in mind the importance of giving users a friendly interface. Primary goal of spend smart application was to reduce the manual efforts made by users to maintain their finance. The uses of shaden components , react for making all interactive elements with the user prespective. Handling the backend job using next js with fast response meeting basic accessibility standards. Tailwind CSS’s responsive utility classes ensure the application functions well on screens ranging from mobile phones to large desktop monitors. The AI-powered receipt scanning feature specifically targets the most friction heavy part of expense tracking data entry and eliminates it through automation, extraction of form fields through AI along with proving insights and suggestion for user finance history every month.

D. Comparison with Existing Systems

Feature	Traditional App	FinSight AI	Spend Smart
Receipt scanning	Basic OCR only	OCR + NLP	Gemini multimodal
Monthly AI reports	Not available	Available	Available with Gemini
Background automation	None	Inngest	Inngest (cron + events)
Authentication	Basic	Clerk	Clerk with MFA
Bot/abuse protection	None	Arcjet	Arcjet middleware
Email communication	Basic	Resend	Resend + React Email
Visualization	Limited	Dashboard charts	Recharts interactive charts
Cloud file storage	None	Supabase	Supabase with signed URLs

E. Limitations and Future Work

Despite its capabilities, Spend Smart currently relies on manual account setup and does not support automatic bank feed integration through systems like Open Banking. it still requires human efforts to add their transaction into the application, user has to upload the digital receipt for scanning purpose. Future work will focus on integrating financial data APIs that allow users to connect their bank accounts directly and handles the online transactions made without any manual efforts. This will enable fully automated transaction imports. The Gemini receipt scanning works well for printed and digital receipts, but it can produce inconsistent results for handwritten receipts. user paying through UPI make it easy for spend smart with literally zero efforts. though it also raise the concern of safety for monitoring the online transaction of the user, it must require the user allowance to monitor, ensuring the user user satisfaction regarding safety. Future versions of Spend Smart will also look into predictive analytics, which uses historical transaction data to forecast future spending and savings patterns. Additionally, there will be a conversational AI interface that lets users ask about their financial data in natural language.

X. CONCLUSION

Spend Smart is an AI-powered expense tracking system that demonstrate how latest web- technologies and intelligent automation can improve personal finance management. Our proposed system Spend Smart integrates a full-stack architecture using Next.js, backend services, and database management to provide a simple and user friendly platform for monitoring income and expenditure. Key features like real-time transaction management, budget monitoring, data visualization through charts and automated alerts improve the usability and effectiveness of the application. Advanced features like AI-based receipt scanning, monthly financial insights generation, and automated email notifications strengthens the system by reducing the manual effort from user’s end and make it more attractive toward users. To ensure data integrity and user privacy, our system Spend Smart also implement security measures like authentication, rate

limiting, and bot protection. Additional features like use of scheduling mechanisms for recurring transactions and budget alerts highlights the capability of system to provide continuous financial assistance.

XI. REFERENCES

- [1]. **Rahmah Mahdi**, “*Personal Expenses Tracker*”, Methodist University, 2024.
DOI: 10.13140/RG.2.2.20455.25760
- [2]. **Sneha Gupta, Unnati Jaiswal, Shubhangi Bhardwaj, Riya Bhargava**, “*Personal Expense Tracker*”, International Journal of Novel Research and Development, Vol. 9, Issue 5, 2024.
- [3]. **Dr. Nidhi Sharma, Dr. Alok Sharma**, “*Smart Personal Expense Tracker Technology*”, International Journal of Research and Analytical Reviews, Vol. 11, Issue 1, 2024.
- [4]. **Tushar Khuteta**, “*Expense Tracker System (ETS)*”, IJPREMS, 2025.
- [5]. **A. Saraboji, Dr. Santhana Lakshmi**, “*Expense Tracker Application*”, IARJSET, Vol. 11, Issue 6, 2024.
- [6]. **Lavesh Lingayat et al.**, “*Design and Implementation of Real-Time Expense Tracker Using Machine Learning*”, ICICC Conference, 2024.
- [7]. **Pooja S. Saravade et al.**, “*Income and Expense Daily Tracker System*”, IJIERT, 2022.
- [8]. **Mohammed Sahel et al.**, “*Streamlining Personal Finances: An Expense Tracker Website Study*”, IEEE SPARC Conference, 2024.
- [9]. **(Multiple Authors)**, “*Expense Tracker using OCR and Random Forest Algorithm*”, IJERST, 2025.
- [10]. **Satyam Jha et al.**, “*Design and Implementation of a Web-Based Expense Tracker System for Personal Finance Management*”, Zenodo, 2025.